Computer Networks 204 (2022) 108719

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

Software Article

NFStream
A flexible network data analysis framework

Zied Aouini [a], Adrian Pekar [b,*]

[a] SoftAtHome, Colombes, France
[b] Department of Networked Systems and Services, Budapest University of Technology and Economics, Hungary

ARTICLE INFO

Keywords:
Traffic flow measurement
Flow features
Framework
Data processing
Data labeling

ABSTRACT

Network traffic analytics have increased in relevance as researchers promoted machine learning techniques to tackle several traffic management challenges. Over the past decade, the research community and the networking industry have investigated, proposed, and developed a growing number of solutions. However, a large subset of proposed approaches is based on unreliable measurement tools and methodologies. Additionally, some findings are reported on private datasets, which results in a lack of applicability and reproducibility. This paper covers the design and implementation of NFStream, a flexible network data analysis framework. Its key features are flexibility, real-time statistical analysis, and the ability to provide reliable ground truth for modern network usage. NFStream provides the community with a common research framework that can help stimulate research in this field and develop more efficient, reproducible solutions.

Code metadata

Current Code version	v6.3.3
Permanent link to code of this code version	https://github.com/ELS-COMNET/COMNET-2021-1130
Legal Code License	GNU Lesser General Public License v3.0
Code Versioning system used	Git
Software Code Language used	Python, C
Compilation requirements, Operating environments & dependencies	cffi>=1.14.0, psutil>=5.7.0, numpy<=1.18.5, pandas>=1.0.3, dpkt>=1.9.4
Developer documentation	www.nfstream.org/docs/
Support email for questions	www.github.com/nfstream/nfstream

1. Introduction

Traffic and services monitoring has always played a strategic role in understanding and managing computer networks. Several methodologies and tools have been engineered over the years to assist daily management routines, obtain critical insight, and help maintain network performance. A comprehensive list of network monitoring tools is provided in [1].

A general observation is that most tools were designed to work on operative networks. Popular flow exporters such as YAF [2], pmacct [3], CAIDA CoralReef [4], SoftFlowd [5], and nProbe [6] are in close conformity with the NetFlow [7]/IPFIX [8] (or sFlow [9]) technologies. However, the deployment of such tools is highly invasive as special flow collectors are typically required to extract and interpret the exported flow records. Furthermore, most tools provide flow measurement only at a coarser granularity or lack functionalities, such as tunnel decoding, application awareness, or capabilities for processing traces in both online (passive packet sniffing) and offline (pcap processing) modes.

Additionally, there is a lack of native integration of Machine Learning (ML) in available flow metering solutions. ML systems learn from

* Corresponding author.
E-mail addresses: zied.aouini@softathome.com (Z. Aouini), apekar@hit.bme.hu (A. Pekar).

https://doi.org/10.1016/j.comnet.2021.108719
Received 13 August 2021; Received in revised form 22 November 2021; Accepted 16 December 2021
Available online 7 January 2022
1389-1286/© 2022 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

empirical data to automatically associate objects with corresponding classes. ML has been deployed to make an operational sense out of the massive data generated by the continuous growth in the number of connected devices and applications. However, although various methods have been developed for multiple use-cases (e.g., traffic prediction [10], traffic classification [11,12], traffic routing [13], congestion control [14], resource management [15], fault management [16], QoS and QoE management [17], and network security [18–20]), ML-based traffic management has not reached a consensus in terms of methodologies and definitions [21]. Consequently, assessing and reproducing state-of-the-art approaches have remained impracticable and challenging [22].

NFStream is an open-source network data analysis framework aimed at filling this gap. Its main goal is to provide the research community with a reliable and extensible framework, facilitating the path from networking to ML. NFStream can stimulate future research in network and services management and provides a basis for developing efficient, reproducible methods.

2. NFStream highlights

NFStream is an open-source framework that allows high throughput network traffic flow analysis to be run on commodity hardware. The flows are being formed by aggregating packets that share a common *key*. A seven-tuple currently defines this *flow key*: the source and destination IP addresses and port numbers, and the protocol, VLAN, and tunnel identifiers. (If tunnel decoding is deactivated, only a six-tuple is used with no tunnel identifier in the flow key.) Traffic information, such as *flow features* (e.g., the total number of bytes of all packets belonging to a particular flow) and the flow key – also often referred to as *flow properties* – is carried in NFStream in *flow entries*. Flow features are derived from the IP, TCP, and UDP packet headers. Their computation involves analysis-centric methods, such as determining the statistical summary (minimum, maximum, mean and standard deviation) of the packet length and inter-arrival time, TCP flags accounting or the early sequence of packets dynamics (sizes, directions and inter-arrival time).

Besides flow statistics calculation, NFStream can also determine the types of applications that generate the flows. This feature – also often referred to as application awareness – is achieved by integrating the nDPI [23] library. nDPI is a ntop [6] maintained superset of the popular OpenDPI library. It detects protocols at the application layer, regardless of the port being used. Furthermore, nDPI can also handle encrypted traffic via its built-in decoder for SSL (both client and server) certificates. nDPI can work even when no complete packet payload is available. Truncating packet payloads is a common technique in monitoring operational network traffic. Because of storage space constraints and privacy reasons, only a limited portion of packet payloads are exposed to the sniffers. NFStream enables accurate traffic flow classification on an application layer since it can handle such cases via nDPI, implying its applicability not only in unsupervised but also supervised ML use cases.

NFStream was designed to operate on both live and offline modes and rely on libpcap, the de facto standard library used for packet capture. High-speed live network traffic measurement is achieved via parallelism, specifically via a modified version of libpcap that supports AF_PACKETv3 (a socket in Linux that allows an application to receive and send raw packets) and allows load balancing packets across several cores on Linux systems. On platforms not supporting AF_PACKETv3, NFStream implements flow-aware hash-based dispatching that runs in both online and offline modes.

Tunneled traffic is typically composed of different flows, but these flows might be organized into one flow from a measurement point of perspective. Decoding tunneled traffic is not an obvious task to implement, mainly due to the variety of protocols. Hence, the majority of flow metering tools do not provide such functionality. Contrarily, NFStream supports tunnel decoding (GTP, CAPWAP, and TZSP). Consequently, it can correctly recognize packets sourcing truly from different flows, reflecting the true characteristics of the traffic, providing thus more accurate flow perception.

NFStream being a highly flexible framework, its capabilities can be easily activated (deactivated), addressing various application use cases with different resource and performance requirements. In addition, it can also be used as a library, allowing simple integration with other tools and services that work in operative networks. Furthermore, NFStream was designed to be extensible, allowing the creation of new features and capabilities within a few lines of Python code. Extensibility use cases include customizing the expiration logic, adding new flow features, and deploying machine learning models on the fly, to name a few.

NFStream also seeks establishing a unified ML ecosystem for building better and sustainable solutions. Each programming language has its set of ML frameworks. However, the Python programming language has become the de facto natural choice of commercial and academic organizations for designing ML-based solutions. This interest is measurable by the broad set of open-source libraries developed in the last decade (i.e., Google TensorFlow [24], Facebook PyTorch [25], Microsoft LightGBM [26], scikit-learn [27]). The deployability of models built on top of these frameworks could be very limited outside the Python ecosystem [28]. NFStream provides a flow-based measurement framework that supports the native interface of the above libraries, establishing a unified ecosystem.

In light of the above-mentioned, NFStream can help accelerating a paradigm shift from data to model/algorithm sharing. Sharing models and their underlying learning algorithms is gaining popularity. Meaningful sharing of data with third-party researchers is limited mainly due to the privacy-sensitive nature of much of today's network data. Where dataset sharing is unavailable, a viable alternative in the context of ML-based networking research is focusing on sharing the learned models and their underlying learning algorithms. However, this alternative is predicated by the requirement for a common ground, a framework such that the shared learning models/algorithms can be trained with the same type of obtained data (features) that researchers can collect in their network environment.

With all that being said, NFStream has its scientific merit as a standard measurement methodology with a broad set of functionalities and native integration of popular Python ML frameworks.

3. High-level description

Fig. 1 depicts the overall architecture of NFStream composed of 2 main components: *NFStreamer* and a set of parallel *Meters*. In what follows, we briefly describe the main functions of these components.

3.1. Packet observation

The packet observation layer is destined for observing packets from both online and offline traffic capture. This layer is implemented in C and bound to Python using C Foreign Function Interface (CFFI) [29], an interface that allows the interaction with almost any C code from Python, to avoid bottleneck issues. This implementation choice allows for performing several packet-related processes efficiently while exposing a unique NFPacket Python object.

1. **Packet capture** is enabled on the network interface card level. After passing various checksum error checks, the packets stored in on-card reception buffers are moved to the hosting device memory. Several libraries are available to capture network traffic. The most popular is perhaps libpcap destined for UNIX-based operating systems, and winpcap for Windows. NFStream implements a modified version of libpcap library that is used for both online and offline modes.

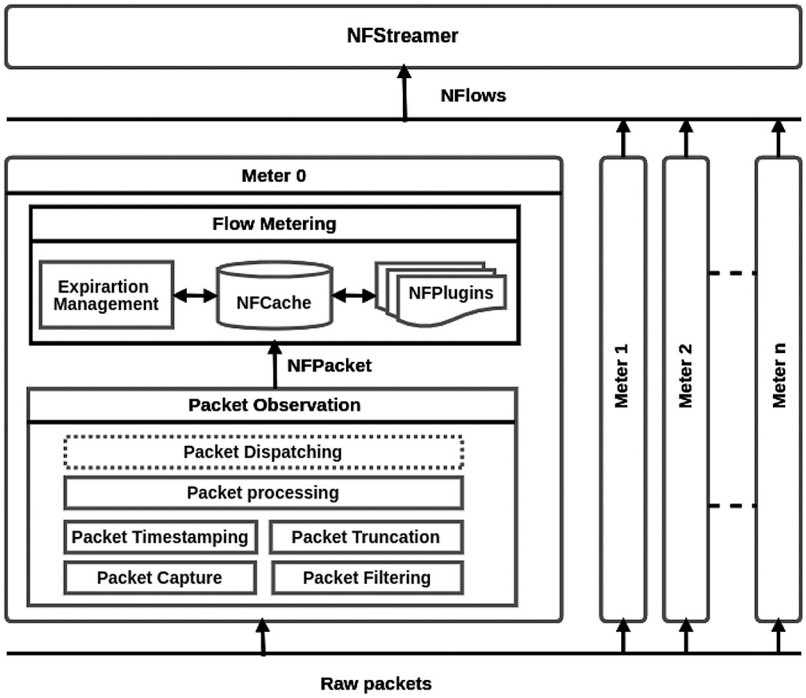


Fig. 1. NFStream overall architecture.

2. **Packet truncation** is destined for selecting precise bytes from the captured packet (e.g., snapshot length). It is also used to reduce the amount of data captured, which leads to reduced CPU and bus bandwidth load.
3. **Packet timestamping** is a mandatory functionality as packets may come from several observation points. NFStream relies on software packet timestamping, which provides milliseconds accuracy.
4. **Packet filtering** serves packet filtering based on a set of characteristics. A packet is selected if the specific fields are equal or in the range of the given values. NFStream packet filtering is based on the Berkeley Packet Filter (BPF) syntax. BPF provides a kernel-based interface to the link and network layers. It possesses features that make it highly efficient at processing and filtering packets [30]. A user-mode interpreter for BPF is provided with the libpcap implementation of the pcap API, so programmers can write applications that transparently support a rich set of constructs to build detailed packet filtering expressions for network protocols.
5. **Packet processing** consists of a set of parsers that allow NFStream to decode the packet and extract its attributes as part of NFPacket object, which is the shared object between the packet observation layer and the metering layer of each meter process. An overview of these attributes is provided in Table 1.
6. **Packet dispatching** consists of load balancing packet processing across parallel meters. As stated above, this feature is implemented via the Linux kernel using the AF_PACKETv3 feature. However, both online mode and offline modes require a load balancing in userspace. NFStream achieves such a task by computing a flow-aware hash for each packet. If the computed hash matches the meter identifier, the packet is consumed. Otherwise, it is used only as a time ticker.

Table 1
Overview of NFPacket attributes.

Name	Description
time	Packet timestamp in milliseconds
delta_time	Delta time in milliseconds with previous flow packet
raw_size	Link layer packet size
ip_size	IP packet size
transport_size	Transport packet size
payload_size	Packet payload size
src_ip	Source IP address string representation
src_mac	Source MAC address string representation
src_oui	Source Organizationally Unique Identifier string representation
dst_ip	Dest. IP address string representation
dst_mac	Dest. MAC address string representation
dst_oui	Dest. Organizationally Unique Identifier string representation
src_port	Transport layer source port
dst_port	Transport layer destination port
protocol	Transport layer protocol identifier
vlan_id	Virtual LAN identifier
ip_version	IP version
ip_packet	Pkt bytes content starting from IP header
direction	Pkt direction defined by metering layer
syn	TCP SYN flag present
cwr	TCP CWR flag present
ece	TCP ECE flag present
urg	TCP URG flag present
ack	TCP ACK flag present
psh	TCP PSH flag present
rst	TCP RST flag present
fin	TCP FIN flag present
tunnel_id	Tunnel identifier

3.2. Flow metering

The flow metering layer implements the flow measurement logic of NFStream. Its primary functions include aggregating packets into flows, flow feature computation, and flow expiration management.

1. **NFCache** stores the entries in a hash map (Python dictionary) and maintains a least recently used list of entries. Flow metering uses these structures to stores information regarding active flows. A flow hash determines whether an NFPacket matches an existing entry or not. In the case of a match, the flow features are updated. Otherwise, a new entry is created and

Table 2
Overview of the extracted flow features.

Category	Name	S2D	D2S	BD	Feature description
Core	id			✓	Flow identifier
	expiration_id			✓	Flow expiration type identifier (e.g., 0 for inactive, 1 for active, and negative for custom)
	src_ip			✓	Flow source IP address string representation
	src_mac			✓	Flow source MAC address string representation
	src_oui			✓	Flow source Organizationally Unique Identifier string representation
	src_port			✓	Flow transport layer source port
	dst_ip			✓	Flow destination IP address string representation
	dst_mac			✓	Flow destination MAC address string representation
	dst_oui			✓	Flow destination Organizationally Unique Identifier string representation
	dst_port			✓	Flow transport layer destination port
	protocol			✓	Flow transport layer protocol identifier
	ip_version			✓	Flow IP version
	vlan_id			✓	Flow Virtual LAN identifier
	first_seen_ms	✓	✓	✓	Timestamp in milliseconds on first flow packet
	last_seen_ms	✓	✓	✓	Timestamp in milliseconds on last flow packet
	duration_ms	✓	✓	✓	Flow duration in milliseconds
	packets	✓	✓	✓	Flow packets accumulator
	bytes			✓	Flow bytes accumulator
Tunnel decoding	tunnel_id	✓	✓	✓	Tunnel identifier
Post-mortem stats	min_ps	✓	✓	✓	Flow minimum packet size
	mean_ps	✓	✓	✓	Flow mean packet size
	stdev_ps	✓	✓	✓	Flow packet size sample standard deviation
	maximum_ps	✓	✓	✓	Flow maximum packet size
	min_piat_ms	✓	✓	✓	Flow minimum packet interarrival time in milliseconds
	mean_piat_ms	✓	✓	✓	Flow mean of packet interarrival time in milliseconds
	stdev_piat_ms	✓	✓	✓	Flow sample standard deviation of packet interarrival time in milliseconds
	maximum_piat_ms	✓	✓	✓	Flow maximum observed packet interarrival time in milliseconds
	syn_packets	✓	✓	✓	Flow cumulative count of packets with TCP SYN flag set
	cwr_packets	✓	✓	✓	Flow cumulative count of packets with TCP CWR flag set
	ece_packets	✓	✓	✓	Flow cumulative count of packets with TCP ECE flag set
	urg_packets	✓	✓	✓	Flow cumulative count of packets with TCP URG flag set
	ack_packets	✓	✓	✓	Flow cumulative count of packets with TCP ACK flag set
	psh_packets	✓	✓	✓	Flow cumulative count of packets with TCP PSH flag set
	rst_packets	✓	✓	✓	Flow cumulative count of packets with TCP RST flag set
	fin_packets	✓	✓	✓	Flow cumulative count of packets with TCP FIN flag set
Early statistics	splt_direction			✓	List of N first flow packet directions (0: src2dst, 1: dst2src, -1:no packet)
	splt_ps			✓	List of N first flow packet sizes (−1 when there is no packet)
	splt_piat_ms			✓	List of N first flow packet inter arrival times (always 0 for first packet, −1 when there is no packet)
Ground-truth	application_name			✓	nDPI application name
	application_category_name			✓	nDPI application category name
	application_is_guessed			✓	Indicates if detection is based on pure dissection or on a port-based guess
	requested_server_name			✓	Requested server name (SSL/TLS, DNS, HTTP)
	client_fingerprint			✓	Client fingerprint (DHCP fingerprint for DHCP, JA3 for SSL/TLS, and HASSH for SSH)
	server_fingerprint			✓	Server fingerprint (JA3 for SSL/TLS and HASSH for SSH)
	user_agent			✓	Extracted user agent for HTTP or User Agent Identifier for QUIC
	content_type			✓	Extracted HTTP content type

S2D = source to destination, D2S = destination to source, BD = bidirectional.

initiated. A flow entry is considered bidirectional if its address-port pair and its reverse belong to the same entry. Table 2 gives a description of the computed 88 flow features categorized as core, tunnel decoding, post-mortem statistics, early statistics, and ground-truth.

2. **Expiration management** runs on top of three flow termination logics. The first one is active expiration and terminates a flow that is active during a predefined period. The second is referred to as inactive expiration. It ends a flow that is being inactive during a predefined period. The last logic represents a custom expiration solution defined by the user at runtime (i.e., flow packets limit).
3. **NFPlugins** are a set of NFPlugin, which is a user-defined extension of NFStream. An NFPlugin is instantiated by using a flexible set of keyword arguments, including specific parameters or external data required for the flow feature computation (i.e., trained model, externally loaded C library).
   The flow metering process calls each NFPlugin defined by the user at mainly three flow existence stages: initiation, update, and expiration. Thus, an NFPlugin defines a method called for each stage. `on_init` method is called for creation with the first packet belonging to it. `on_update` is triggered each time a new NFPacket is mapped to the flow entry. Finally, `on_expire` is performed when at entry is considered as expired. Consequently, extending NFStream is simple. Adding new flow features or ML model outcomes can be achieved in just a few lines. Listing 1 demonstrates such a scenario where a trained ML object (i.e., trained scikit-learn model object) is evaluated in real-time. For brevity, in this example, we suppose that the model was trained based on per-flow bidirectional bytes and packet counters.

3.3. Flow export

The export layer is implemented as part of the NFStreamer class. NFStreamer is the main class of the NFStream framework. It is responsible for setting the overall workflow, mainly the orchestration of parallel

Listing 1: Example of extending NFStream with an ML model prediction.

```
from nfstream import NFStreamer
import numpy

class ModelPrediction(NFPlugin):
    def on_init(self, packet, flow):
        flow.udps.model_prediction = 0
    def on_expire(self, flow):
        # You can do the same in on_update entry point and force expiration
              with custom id.
        to_predict = numpy.array([flow.bidirectional_packets,
                                  flow.bidirectional_bytes]).reshape((1,-1))
        flow.udps.model_prediction = self.my_model.predict(to_predict)

ml_exp = NFStreamer(source="eth0", udps=ModelPrediction(my_model=
    model))

for flow in ml_exp:
    print(flow.udps.model_prediction)
```

Table 3
Configuration options for the metering and observation processes.

Argument	Description
`source`	Packet capture source. Pcap file path or network interface name
`decode_tunnels`	Enable/Disable GTP/CAPWAP/TZSP tunnels decoding
`bpf_filter`	Specify a BPF filter for filtering selected traffic
`snapshot_length`	Control packet slicing size (truncation) in bytes
`idle_timeout`	Flows that are idle (no packets received) for more than this value in seconds are expired
`active_timeout`	Flows that are active for more than this value in seconds are expired
`accounting_mode`	Specify the accounting mode that will be used to report bytes related features (0: Link layer, 1: IP layer, 2: Transport layer, 3: Payload)
`udps`	Specify user defined NFPlugins used to extend NFStreamer
`n_dissections`	Number of per flow packets to dissect for L7 visibility feature
`statistical_analysis`	Enable/Disable post-mortem flow statistical analysis
`splt_analysis`	Specify the sequence of first packets length for early statistical analysis
`n_meters`	Specify the number of parallel metering processes. When set to 0, NFStreamer will automatically scale metering according to available physical cores on the running host
`performance_report`	Performance report interval in seconds. Disabled when set to 0 and ignored for offline mode

metering processes and the definition of the flow export format. Thus, working with flow-based data is as simple as instantiating a single class. Listing 2 illustrates the creation and usage of the NFStreamer object. NFStreamer is highly configurable and provides an extensive set of arguments for controlling both the metering and observation processes, as shown in Table 3.

NFStreamer methods define the export format of the measured flows. While it is possible to iterate over the NFStreamer object, methods include CSV file and pandas [31] dataframe conversions. Selecting pandas format came as natural as it is the de facto standard input format for ML frameworks. Finally, the conversion process supports features anonymization based on the Blake2 [32] algorithm.

4. Impact overview

Network traffic flow measurement has been a subject of several research studies and various tools have been developed for multiple use-cases over the last decades. However, existing solutions typically offer only a set of functionalities close to NFStream core features. Tunnel

Listing 2: Iterating over flows, converting them into Pandas dataframe, and exporting into a CSV.

```
from nfstream import NFStreamer

exp = NFStreamer(source=pcap_file_or_live_interface_name,
                 decode_tunnels=True,
                 bpf_filter=None,
                 promiscuous_mode=True,
                 snapshot_length=1536,
                 idle_timeout=120,
                 active_timeout=1800,
                 accounting_mode=0,
                 udps=None,
                 n_dissections=20,
                 statistical_analysis=False,
                 splt_analysis=0,
                 n_meters=0,
                 performance_report=0)

# Iterate over flows
for flow in exp:
    print(flow)

# Convert the flows into a Pandas dataframe:
my_dataframe = exp.to_pandas(columns_to_anonymize=[])
my_dataframe.head()

# Export flows into a CSV File:
total_rows = exp.to_csv( path=None,
                         flows_per_file=0,
                         columns_to_anonymize=[])
```

decoding, application awareness, or capabilities for processing traces in both online (passive packet sniffing) and offline (pcap processing) modes are functionalities that are typically absent in present solutions, especially their simultaneous presence.

TiE [33] is a noteworthy solution, however, its ground-truth development is based on unreliable methods. Specifically, it utilizes labeling introduced by Moore and Papagiannaki [34] extended with CoralReef and L7-filter that were shown to perform poorly in determining application classes [35,36]. Additionally, the integration of ML classifiers relies on a different platform (e.g., Weka [37]). Moreover, it also lacks the option of adding flow features on the fly. Lastly, the source code is made available only at request, potentially leading to fewer open-source community contributions. In contrast, NFStream uses nDPI for ground truth development, a library that is thoroughly maintained and shown to perform reliably [38]. Furthermore, NFStream integrates ML effectively via its platform-independent NFPlugin that, besides ML integration, also makes it highly extensible with new features. Lastly, the superiority of NFStream over TiE is also manifested in its quality of being an open-source solution.

Most open-source flow metering tools do not provide performance indicators. Even though it is specified in cases, it is usually a performance indicator of what the developers achieved on test systems rather than a guaranteed performance [39]. This is mainly because the performance of flow meters strongly depends on several factors such as the number of packets to be processed, the volume of flows, and the performance and capabilities of the underlying system (i.e., CPU, memory, storage technology, and network interface card). Parallel processing also significantly impacts performance, although multi-thread (CPU) processing of the captured packets is not common to see among existing open-source solutions. Lastly, the number of calculated flow features (statistics) and their computing complexity (requirements) also heavily impact the processing time.

The latter two factors exceptionally hindered the comparison of NFStream to popular solutions such as YAF [2], pmacct [3], CAIDA CoralReef [4], SoftFlowd [5], and nProbe [6]. These tools can calculate a significantly lower number of flow features and, in doing so, provide flow measurement only at a coarser granularity. Additionally, these solutions are in close conformity with the NetFlow/IPFIX

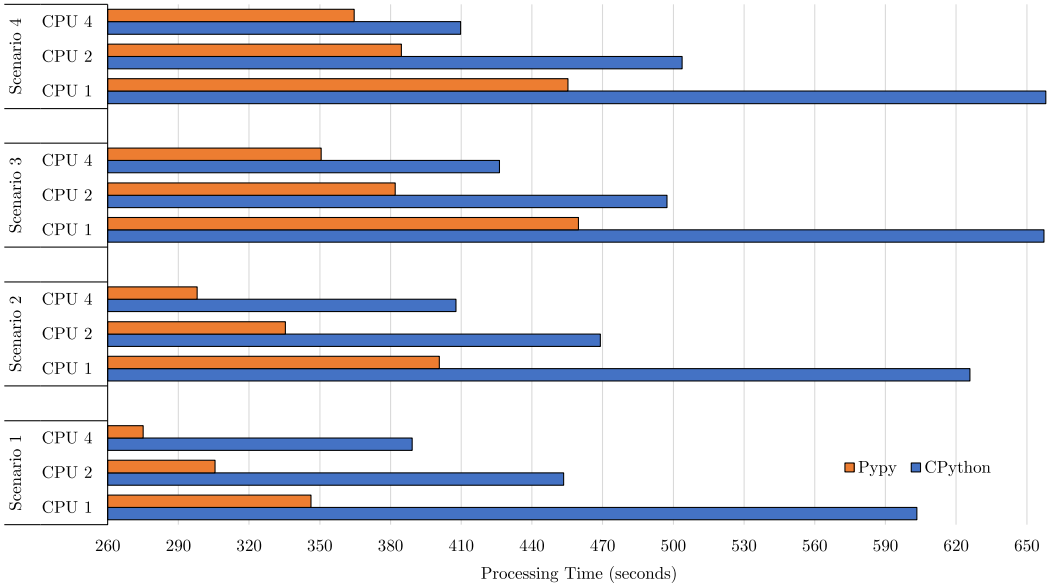


Fig. 2. CPython vs. PyPy NFStream processing times relative to the number of used cores.

Table 4
Benchmark evaluation scenarios.

Scenario	CF	TD	GTF	SF	SP	AF	NCF
1	✓						28
2	✓	✓	✓				37
3	✓	✓	✓	✓	✓		88
4	✓	✓	✓	✓	✓	✓	88

CF = Core Features, TD = Tunnel Decoding, GTF = Ground-truth, SF = Statistical Features, SP = Splt Features, AF = Anonymization, NCF = Number of Computed Features.

(or sFlow) technologies, making their deployment highly invasive as special flow collectors are required to extract and interpret the exported flow records. NFStream, instead, was designed to provide the measured flow-level information in a variety of file formats, such as Python Array and Pandas DataFrame, or export them directly into CSV for immediate model training without the need for additional preprocessing. Besides the already supported formats, NFStream allows new formats to be easily integrated thanks to its flexible design. As a result, emerging technologies such as event streaming via Apache Kafka can be implemented in just a few lines of code.

4.1. NFStream Performance Indicators

An unbiased comparison with existing solutions is undoubtedly complex and prone to various factors that can distort actual efficacy. Nonetheless, we performed a benchmark of processing times across multiple parameter settings to give a grasp on NFStream's performance.

To this end, we organized the packets of a pcap file into flows using the default configuration of NFStream and stored the obtained flow entries in a CSV file (cf. Listing 2). The pcap used for the benchmark was captured at an average-sized European university network. The average traffic rate of the measured uplink was around 1.60 Gbps. The capture duration of the traffic trace is 299 s, and it contains 62 M packets with complete payloads. The data byte and bit rates of the trace are 165 MBps and 1320 Mbps, respectively, with an average packet size of 795.35 bytes and an average packet rate 207 kpackets/s. It is noteworthy that the monitored network has not reached its typical traffic rate due to the COVID-19 pandemic.

The evaluation ran on a system with four Intel(R) Xeon(R) Silver 4215 CPUs @ 2.50 GHz. We systematically increased the number of activated features of NFStream, as shown in Table 4, and measured the processing time relative to the number of cores involved in processing. The obtained results are depicted in Fig. 2 and demonstrate a significant speed gain when running NFStream on top of PyPy vs. CPython. In scenarios 1 and 2, the average speed gain of PyPy vs. CPython was around 34%. Tunnel decoding and determining application types (ground truth) seem to have little to no impact on top of the core functionality demands of NFStream. In scenarios 3 and 4, the speed gain is lower. With only one CPU core involved in the processing, the average speed gain was around 30% in scenarios 3 and 4, while with two CPU cores, the gain dropped to around 23%. Interestingly, the lowest speed gain of PyPy vs. CPython was achieved using 4 CPU cores, specifically around 17% in scenario 3 and around 11% in scenario 4. However, neither the activation of statistical features nor splt features or anonymization have a significant overhead on the processing time of NFStream. Our results showed only around a 9% increase of all features activated vs. only core features. Overall, PyPy has a positive impact on the operation of NFStream. CPython compiles the Python code into bytecode and interprets it within the evaluation loop, which results in less efficiency than other compiled languages. In contrast, PyPy leverages Just In Time (JIT) compilation (translating a subset of Python code into fast machine code) leading to significant performance gain. NFStream also leverages CFFI bindings helping to improve its performance further. It is noteworthy that the performance overhead and demands can be reduced further by selecting only a subset of discriminators (features) required for a particular use case.

While there is no tool with a comparable set of functionalities, we provide processing times of two moderately related tools to give a better sense of the achieved performance. CICFlowMeter [40,41] is a popular tool in network security and anomaly detection. In our evaluation, we first attempted to process (on the same system) the pcap file used for NFStream benchmarking while setting CICFlowMeter v4 using its default configuration. However, we, unfortunately, could not obtain the processing time of CICFlowMeter due to a Java heap space error. The tool crashed after 39 min. Using a smaller PCAP file containing only 10 M packets with complete payloads, we eventually managed to obtain the processing time: 253 s (CICFlowMeter) vs. 60 s (NFStream running on top of Pypy and all features being activated). CICFlowMeter measured 84 features per 96 458 flows. In contrast, NFStream collected (91 759) 92 117 flows with (de)activated tunnel decoding. Upon close examination of the results, we found that the difference between CICFlowMeter and NFStream flow counts was due to the different flow expiration timers (NFStream idle/active

timeout defaults are 120/1800 s while CICFlowMeter defaults are hard-coded 120/5 s) and the IPv6 flow processing functionality (by default NFStream measures IPv6 flows while CICFlowMeter does not). However, the poor documentation and the hard-coded manner in which CICFlowMeter was developed made any further investigation of this difference impractical.

Similarly, we also compared NFStream to ndpiReader [42], a tool purely written in C. ndpiReader is an example implementation of nDPI [23] that, besides basic flow statistics collection, demonstrates the application awareness capabilities of the library. ndpiReader processed the pcap file with 62 M packets in around 413 s. In contrast, our tool written in Python not only could process this file in a shorter duration, but it did so with a significantly finer granularity. However, it is essential to emphasize that both CICFlowMeter and ndpiReader were not developed to be run on multiple CPU cores. Thus, their achieved results should be interpreted with caution to avoid drawing debunk conclusions.

4.2. Scholarly Publications Enabled by NFStream

The popularity of NFStream is characterized by a steady increase. Castaneda Herrera et al. [43] developed a novel approach for identifying video streaming services in 5G Networks. The flow records classified via supervised machine learning (including Logistic Regression, KNN, Naive Bayes, and Regression Tree) are obtained using NFStream. Liu et al. [44] proposed a lightweight hybrid form of IDS — an embedded model for feature selection and a convolutional neural network for attack detection and classification. Using NFStream, the Authors collected and developed the CCD-INID-V1 dataset, whose traffic was captured at the Center for Cyber Defense, North Carolina A&T State University. Bikmukhamedov and Nadeev [45,46] introduced a neural network framework that allows constructing multi-class network traffic models suitable for flow generation and classification tasks. During the evaluation, they utilized NFStream to assign flow labels and extract packet features from pcap files. Sun et al. [47] proposed a deep learning approach to detect malware using data collected from a web crawler that systematically sent requests to benign and malicious websites on the Internet. NFStream was used to perform traffic flow analysis. The work was developed as part of a research project aimed at enhancing the defensibility of network systems against malware, AI-based cyber attacks, and other security threats. Jonsson and Edeby [48] analyzed Tor network traffic that reveals what data is sent through the network. The data collected using NFStream at three Tor exit nodes (US, Germany, and Japan) helped drawing conclusions about Tor usage. Finally, Pekar et al. [49] studied four publicly available traffic traces (UNIV1, UNIV2, CAIDA2016, and CAIDA2018) and provided useful insights and suggestions on how to determine more justified and valid thresholds for heavy-hitter network traffic flow detection. The traffic traces were processed and organized into flow records using NFStream.

5. Conclusion

This paper introduced NFStream, an open-source network data analysis framework. Its parallel processing capabilities, online and offline traffic processing modes, application awareness, flexibility, and straightforward path from networking to ML make NFStream viable among existing open-source solutions. By leveraging the tool, research outcomes can become more natural to reproduce, comparable against other studies, and generalizable to other systems that have not been studied yet.

While the obtained results are promising, our work revealed both challenges and space for improvements. We actively work on increasing the performance and flexibility of NFStream. We plan to extend its support for Microsoft Windows operating systems. We also consider the implementation of high-speed packet capture frameworks, such as Intel DPDK [50]. Finally, a consistent effort will also be put into dataset development. We believe they can provide an excellent incentive for much broader adoption of both ML/AI and reproducibility.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Les Cottrell, Network monitoring tools, 2021, GitHub Repository, SLAC National Accelerator Laboratory, https://www.slac.stanford.edu/xorg/nmtf/nmtf-tools.html.

[2] NetSA, YAF, 2021, GitHub Repository, Carnegie Mellon University, SEI, CERT, https://tools.netsa.cert.org/yaf/index.html.

[3] pmacct, Pmacct, 2021, GitHub Repository, pmacct, http://www.pmacct.net.

[4] CAIDA, CoralReef, 2021, GitHub Repository, CAIDA, https://www.caida.org/tools/measurement/coralreef/.

[5] D. Miller, H. Irino, SoftFlowd, 2021, GitHub Repository, SoftFlowd, https://github.com/irino/softflowd.

[6] nTop, 2021, High Performance Network Monitoring Solutions.

[7] B. Claise, Cisco systems NetFlow services export version 9, in: Request for Comments, (3954) 2004, http://dx.doi.org/10.17487/RFC3954.

[8] P. Aitken, B. t Claise, B. Trammell, Specification of the IP flow information export (IPFIX) protocol for the exchange of flow information, 2013, http://dx.doi.org/10.17487/RFC7011, Request for Comments, 7011, RFC Editor, RFC 7011.

[9] S. Panchen, N. McKee, P. Phaal, Inmon corporation's sflow: A method for monitoring traffic in switched and routed networks, 2001, http://dx.doi.org/10.17487/RFC3176, Request for Comments, 3176, RFC Editor, RFC 3176.

[10] P. Poupart, Z. Chen, P. Jaini, F. Fung, H. Susanto, Y. Geng, L. Chen, K. Chen, H. Jin, Online flow size prediction for improved network routing, in: 2016 IEEE 24th International Conference on Network Protocols, ICNP, IEEE, 2016, pp. 1–6.

[11] Z. Aouini, A. Kortebi, Y. Ghamri-Doudane, I.L. Cherif, Early classification of residential networks traffic using C5. 0 machine learning algorithm, in: 2018 Wireless Days, WD, IEEE, 2018, pp. 46–53.

[12] N. Jing, M. Yang, S. Cheng, Q. Dong, H. Xiong, An efficient SVM-based method for multi-class network traffic classification, in: 30th IEEE International Performance Computing and Communications Conference, IEEE, 2011, pp. 1–8.

[13] Z. Lin, M. van der Schaar, Autonomic and distributed joint routing and power control for delay-sensitive applications in multi-hop wireless networks, IEEE Trans. Wireless Commun. 10 (1) (2010) 102–113.

[14] I. El Khayat, P. Geurts, G. Leduc, Enhancement of TCP over wired/wireless networks with packet loss classifiers inferred by supervised learning, Wirel. Netw. 16 (2) (2010) 273–290.

[15] N. Baldo, P. Dini, J. Nin-Guerrero, User-driven call admission control for VoIP over WLAN with a neural network based cognitive engine, in: 2010 2nd International Workshop on Cognitive Information Processing, IEEE, 2010, pp. 52–56.

[16] J.S. Baras, M. Ball, S. Gupta, P. Viswanathan, P. Shah, Automated network fault management, in: MILCOM 97 MILCOM 97 Proceedings, Vol. 3, IEEE, 1997, pp. 1244–1250.

[17] E. Demirbilek, J.-C. Grégoire, Machine learning–based parametric audiovisual quality prediction models for real-time communications, ACM Trans. Multimedia Comput. Commun. Appl. (TOMM) 13 (2) (2017) 1–25.

[18] G. Giacinto, R. Perdisci, M. Del Rio, F. Roli, Intrusion detection in computer networks by a modular ensemble of one-class classifiers, Inf. Fusion 9 (1) (2008) 69–82.

[19] W. Hu, W. Hu, S. Maybank, Adaboost-based algorithm for network intrusion detection, IEEE Trans. Syst. Man Cybern. B 38 (2) (2008) 577–583.

[20] Y. Li, R. Ma, R. Jiao, A hybrid malicious code detection method based on deep learning, Int. J. Secur. Appl. 9 (5) (2015) 205–216.

[21] A. Dainotti, A. Pescape, K.C. Claffy, Issues and future directions in traffic classification, IEEE Netw. 26 (1) (2012) 35–40.

[22] R. Boutaba, M.A. Salahuddin, N. Limam, S. Ayoubi, N. Shahriar, F. Estrada-Solano, O.M. Caicedo, A comprehensive survey on machine learning for networking: evolution, applications and research opportunities, J. Internet Serv. Appl. 9 (1) (2018) 16.

[23] nTop, nDPI, 2021, nTop, https://www.ntop.org/products/deep-packet-inspection/ndpi/.

[24] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, et al., Tensorflow: A system for large-scale machine learning. in: 12th $\{USENIX\}$ Symposium on Operating Systems Design and Implementation, $\{OSDI\}$ 16, 2016, pp. 265–283.

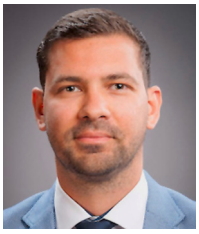
[25] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., PyTorch: An imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, 2019, pp. 8024–8035.

[26] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, Lightgbm: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems, 2017, pp. 3146–3154.

[27] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, et al., Scikit-learn: Machine learning in python, J. Mach. Learn. Res. 12 (2011) 2825–2830.
[28] A. Kortebi, Z. Aouini, C. Delahaye, J.-P. Javaudin, Y. Ghamri-Doudane, A platform for home network traffic monitoring, in: 2017 IFIP/IEEE Symposium on Integrated Network and Service Management, IM, IEEE, 2017, pp. 895–896.
[29] A. Rigo, M. Fijalkowski, CFFI documentation, 2015, URL https://cffi.readthedocs.io/en/latest/.
[30] S. McCanne, V. Jacobson, The BSD packet filter: A new architecture for user-level packet capture, in: Proceedings of the USENIX Winter 1993 Conference Proceedings on USENIX Winter 1993 Conference Proceedings, in: USENIX, vol. 93, 1993, p. 2.
[31] W. McKinney, et al., pandas: a foundational Python library for data analysis and statistics, in: Python for High Performance and Scientific Computing, Vol. 14, no. 9, 2011.
[32] J.-P. Aumasson, S. Neves, Z. Wilcox-O'Hearn, C. Winnerlein, Blake2: simpler, smaller, fast as MD5, in: International Conference on Applied Cryptography and Network Security, Springer, 2013, pp. 119–135.
[33] A. Dainotti, W. de Donato, A. Pescape, G. Ventre, TIE: A Community-Oriented Traffic Classification Platform, Technical Report, (TR-DIS-10-2008) University of Napoli "Federico II", 2008.
[34] A.W. Moore, K. Papagiannaki, Toward the accurate identification of network applications, in: C. Dovrolis (Ed.), Passive and Active Network Measurement, Springer Berlin Heidelberg, Berlin, Heidelberg, 2005, pp. 41–54.
[35] T. Bujlow, V.C.-E. nol, P. Barlet-Ros, Independent comparison of popular DPI tools for traffic classification, Comput. Netw. 76 (2015) 75–89, http://dx.doi.org/10.1016/j.comnet.2014.11.001.
[36] V. Carela-Español, T. Bujlow, P. Barlet-Ros, Is our ground-truth for traffic classification reliable? in: M. Faloutsos, A. Kuzmanovic (Eds.), Passive and Active Measurement, Springer International Publishing, Cham, 2014, pp. 98–108.
[37] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, I.H. Witten, The WEKA data mining software: an update, ACM SIGKDD Explor. Newsl. 11 (1) (2009) 10–18.
[38] L. Deri, M. Martinelli, T. Bujlow, A. Cardigliano, nDPI: Open-source high-speed deep packet inspection, in: 2014 International Wireless Communications and Mobile Computing Conference, IWCMC, 2014, pp. 617–622, http://dx.doi.org/10.1109/IWCMC.2014.6906427.
[39] R. Hofstede, P. Čeleda, B. Trammell, I. Drago, R. Sadre, A. Sperotto, A. Pras, Flow monitoring explained: From packet capture to data analysis with NetFlow and IPFIX, IEEE Commun. Surv. Tutor. 16 (4) (2014) 2037–2064, http://dx.doi.org/10.1109/COMST.2014.2321898.
[40] G. Draper-Gil., A.H. Lashkari., M.S.I. Mamun, A.A. Ghorbani, Characterization of encrypted and VPN traffic using time-related features, in: Proceedings of the 2nd International Conference on Information Systems Security and Privacy- Vol. 1, ICISSP, SciTePress, INSTICC, 2016, pp. 407–414, http://dx.doi.org/10.5220/0005740704070414.
[41] A.H. Lashkari., G.D. Gil, M.S.I. Mamun, A.A. Ghorbani, Characterization of tor traffic using time based features, in: Proceedings of the 3rd International Conference on Information Systems Security and Privacy- Vol. 1, ICISSP, SciTePress, INSTICC, 2017, pp. 253–262, http://dx.doi.org/10.5220/0006105602530262.
[42] nTop, Ndpireader, 2021, GitHub Repository, nTop, https://github.com/ntop/nDPI/blob/dev/example/ndpiReader.c.
[43] L.M. Castaneda Herrera, A. Duque Torres, W.Y. Campo Munoz, An approach based on knowledge-defined networking for identifying video streaming flows in 5G networks, IEEE Lat. Am. Trans. 19 (10) (2021) 1737–1744, http://dx.doi.org/10.1109/TLA.2021.9477274.
[44] Z. Liu, N. Thapa, A. Shaver, K. Roy, M. Siddula, X. Yuan, A. Yu, Using embedded feature selection and CNN for classification on CCD-INID-V1—A new IoT dataset, Sensors 21 (14) (2021) http://dx.doi.org/10.3390/s21144834.
[45] R.F. Bikmukhamedov, A.F. Nadeev, Multi-class network traffic generators and classifiers based on neural networks, in: 2021 Systems of Signals Generating and Processing in the Field of on Board Communications, 2021, pp. 1–7, http://dx.doi.org/10.1109/IEEECONF51389.2021.9416067.
[46] R. Bikmukhamedo, A. Nadeev, Generative transformer framework for network traffic generation and classification, T-Comm 14 (2020) 64–71, http://dx.doi.org/10.36724/2072-8735-2020-14-11-64-71.
[47] Y. Sun, N. Chong, H. Ochiai, Network Flows-Based Malware Detection Using A Combined Approach of Crawling And Deep Learning, in: IEEE International Conference on Communications, 2021, pp. 1–7.
[48] T. Jonsson, G. Edeby, Collecting and analyzing tor exit node traffic, 2021, p. 62, Blekinge Institute of Technology, Faculty of Computing, Department of Computer Science.
[49] A. Pekar, A. Duque-Torres, W.K.G. Seah, O. Caicedo, Knowledge discovery: Can it shed new light on threshold definition for heavy-hitter detection? J. Netw. Syst. Manage. 29 (3) (2021) 24, http://dx.doi.org/10.1007/s10922-021-09593-w.
[50] I. Cerrato, M. Annarumma, F. Risso, Supporting fine-grained network functions through intel DPDK, in: 2014 Third European Workshop on Software Defined Networks, IEEE, 2014, pp. 1–6.



Zied Aouini received the Ph.D. degree in computer science at the University of La Rochelle, La Rochelle, France, in 2017. Currently, he is a senior network data scientist at SoftAtHome (Orange Group), Paris, France. His research interests include machine learning and its numerous applications to network and services management and distributed systems.



Adrian Pekar received the Ph.D. degree in computer science from the Technical University of Košice, Slovakia, in 2014. Currently, he is an Assistant Professor with the Department of Networked Systems and Services, Budapest University of Technology and Economics, Hungary. Prior to this, he held research, teaching, and engineering positions in New Zealand and Slovakia. His research interests include network traffic classification and management, and stream processing.